

演讲简易版（提纲 + 关键词，答辩时用这个）

用法：打印这份提纲，答辩时放在手边。每个模块只记关键词，用自己的话讲。

演讲部分（10-12 分钟）

开场（30秒）

- Qt 6 + C++17 桌面应用
- 15200 行 / 151 文件 / 766 行 MainWindow
- 学业发展规划：课程、目标、简历、AI 分析

一、为什么做？（1分钟）

- 课程多、目标散、简历每次重写
- 需求：7 类数据管理 + GPA 分析 + 简历生成 + AI 建议

二、怎么设计的？（3分钟）重点

四层架构：

UI(Pages/Widgets/Dialogs + 4个协调器)
→ Service(13个) → DAO(10个) → Model(11个) → SQLite

MainWindow 的角色 = "指挥官"，不干活：

- AppShellController → 管页面切换 (QStackedWidget)
- DataRefreshCoordinator → 管跨页刷新 (信号槽)
- BackendRuntimeController → 管后端进程 (QProcess)
- AiContextMediator → 管 AI 上下文 (eventFilter)

核心设计决策： QMap<int, std::function> 注册表替代 switch-case

三、现场演示（3-4分钟）最关键

演示顺序（提前跑通 3 遍）：

1. 侧边栏折叠动画（`QPropertyAnimation` + `QParallelAnimationGroup`）
2. 课程页新增一门课 → 切总览页看 GPA 变化（数据联动）
3. 分析页 → 学期分析 + 同学对标
4. 简历页 → 拖拽 `QSplitter` → 导出 HTML
5. AI 划词提问 → 选中文字 → 浮窗 → 发送

Demo 失败预案：准备 2 分钟录屏视频

四、技术难点（3分钟）

难点 1：5000 行 → 766 行重构

- 单一职责原则，拆成 4 个协调器
- 关键词：解耦、可维护、职责边界

难点 2：跨页数据一致性

- `DataDomain` 枚举 + 信号槽自动刷新
- 关键词：星型拓扑、迪米特法则

难点 3：AI 双模式 + 优雅降级

- 远程不可用 → 自动切本地规则引擎
- 关键词：`QNetworkAccessManager` + 缓存 15 秒 + 超时 2 秒

难点 4：`eventFilter` 全局文本选中

- 根 `Widget` 装 `eventFilter`，拦截所有子控件鼠标事件
- 关键词：`qobject_cast` 逐类型检测 + `manhattanLength` 判断拖拽

五、亮点总结（1分钟）

- 7 种设计模式的实际应用
- Qt 全栈：UI + 动画 + 数据库 + 网络 + 事件系统
- 工程化：从 5000 行重构到 766 行

Q&A 部分（关键词速查）

必问题：UI 线程阻塞

问：QEventLoop 阻塞 UI？

答：承认是技术妥协 → 有 2 秒超时保护 → 优化方案：异步信号槽 / QtConcurrent::run()

必问题：内存管理

答：

- Qt 对象树（new SidebarWidget(this) → 父对象销毁自动回收）
- deleteLater()：用于 QNetworkReply、动画组、浮窗、Toast
- 没用 unique_ptr（Qt 官方建议不要混用）

敏感题：AI 使用

话术："原型驱动 + AI 辅助生成"

- AI 写什么：DAO SQL / UI 布局（样板代码）
- 我写什么：协调器设计 / eventFilter / 策略模式 / 重构
- 三层验证：编译 → 运行时 → Code Review

不要说："让 AI 复制改写"

要说："我手写模板类 → AI 批量生成 → 我 Code Review"

架构题：协调器 vs 中介者

- 协调器 = 消息路由器（数据变了 → 谁需要知道）
- 中介者 = 协议转换器（鼠标事件 → AI 上下文）
- 为什么不直接连信号槽？12 个页面两两连接 = 132 条线

架构题：eventFilter vs 继承

- 控件基类各不相同（QLabel / QListWidget / QTableWidgetItem）
- eventFilter 装一次拦截所有，包括动态添加的子控件
- 还处理了 QEvent::ChildAdded

细节题：信号槽连接类型

- 当前全在主线程 = `DirectConnection`
- 跨线程自动变 `QueuedConnection`（线程安全）
- `deleteLater()` = `DeferredDelete` 事件（不在调用链中立即删）

细节题：QSettings vs SQLite

- `QSettings` = 配置（窗口位置、学生信息、头像路径）→ 注册表
- `SQLite` = 业务数据（课程、目标、经历）→ `pdp.db`

细节题：SQLite 并发

- 当前单用户场景，问题不大
- 优化方案：WAL 模式 / 事务 / 命名连接
- 项目中只设了 `PRAGMA foreign_keys = ON`（如实说）

压力题：项目不足

1. 主线程同步网络（应该异步）
2. 单元测试覆盖不够
3. 没用 QML（Widgets 动画有限）

压力题：加一个新表要多久？

- 3-5 分钟
- 前 4 步 AI 生成（Model/DAO/Service/Dialog）
- 后 2 步手动集成（Page/DataRefreshCoordinator）

关键 Qt 类速查（万一被指着代码问）

你在代码里写的	背后是什么
<code>new QPropertyAnimation(this, "minimumWidth")</code>	Qt 属性动画系统，对任何 <code>QObject</code> 属性做动画
<code>QParallelAnimationGroup</code>	多动画同时播放
<code>QEasingCurve::InOutQuad</code>	先慢后快再慢的缓动

你在代码里写的	背后是什么
<code>installEventFilter(this)</code>	在目标对象上装事件拦截器
<code>qobject_cast<QLabel*>(w)</code>	Qt 的安全类型转换（基于元对象系统）
<code>QSqlDatabase::database()</code>	获取默认数据库连接
<code>QSqlQuery::prepare/bindValue/exec</code>	参数化 SQL 防注入
<code>QNetworkAccessManager::get/post</code>	HTTP 客户端（AI 服务调用）
<code>QSettings</code>	跨平台键值存储（Win=注册表）
<code>QSystemTrayIcon</code>	系统托盘图标
<code>QStackedWidget</code>	层叠页面切换容器
<code>QSplitter</code>	可拖拽分割面板
<code>deleteLater()</code>	事件循环空闲时删除（线程安全）
<code>connect(A, &A::sig, B, &B::slot)</code>	类型安全的信号槽连接

演示前 Checklist

- ☐ `seed_db.py` 已填充测试数据
- ☐ AI Server 已启动（或确认切到本地模式）
- ☐ 录屏视频已准备好（2 分钟）
- ☐ 每个演示步骤跑通过一遍
- ☐ 简历导出的 HTML 文件路径记得
- ☐ 数据库文件 `pdp.db` 位置记得（万一要直接查）

一句话记住：我设计骨架，AI 填充血肉，重构是灵魂。